

Linear Models

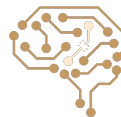
Arash Marioryad

10 Khordad 1405



CE 417-Artificial Intelligence, Sharif Univ. of Tech.

Most Slides have been adopted from Berkeley CS188, CMU & Sharif UT CE417



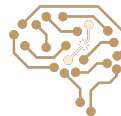
Types of ML problems

- Supervised learning
 - Given: training data + desired outputs (labels/targets)
- Unsupervised learning
 - Given: training data (without desired outputs)
- Reinforcement learning
 - Rewards from sequence of actions



Components of Supervised Learning

- Input space: $\mathcal{X}$
- Output space: $\mathcal{Y}$
- Unknown target function: $f: \mathcal{X} \rightarrow \mathcal{Y}$
- Training data: $(x_1, y_1), (x_2, y_2), \dots, (x_N, y_N)$
- Pick a formula $g: \mathcal{X} \rightarrow \mathcal{Y}$ that approximates the target function f
- Hypothesis Set $\mathcal{H}$: a set (class) of functions that g comes from



Supervised Learning: Regression vs. Classification

Regression: predict a continuous target variable

E.g., $y \in [0,1]$

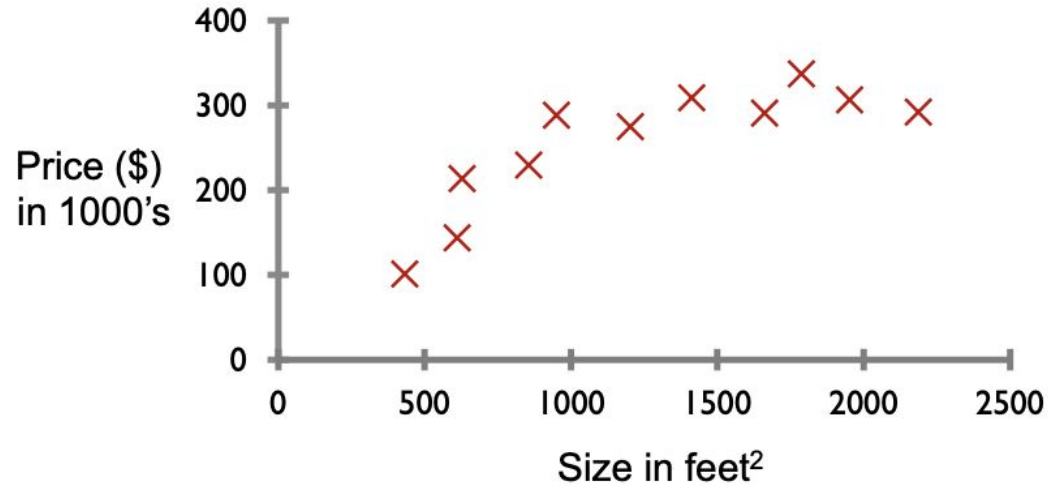
Classification: predict a discrete target variable

E.g., $y \in \{1,2,\dots,C\}$

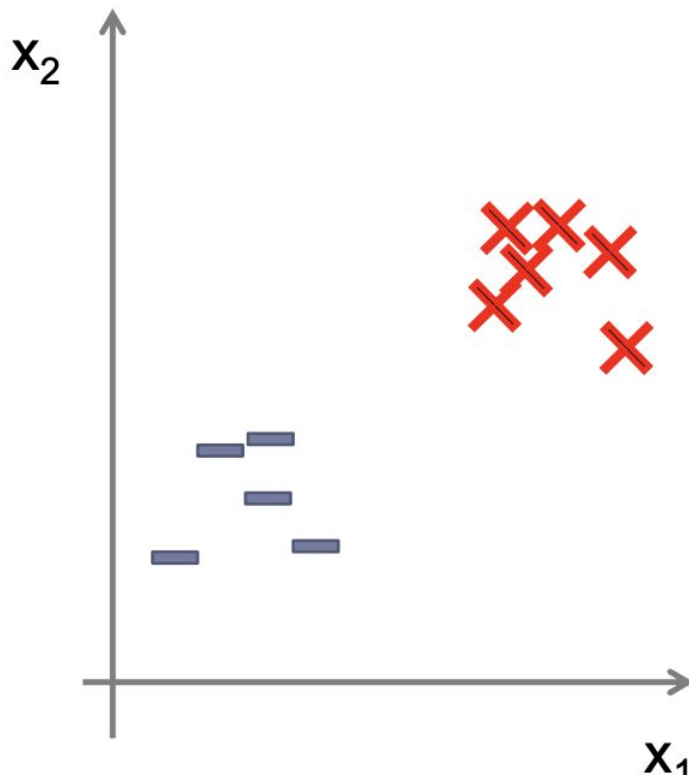


Regression Example

- Housing price prediction

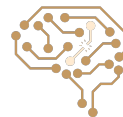


Classification Example (cont.)

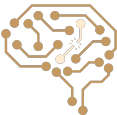
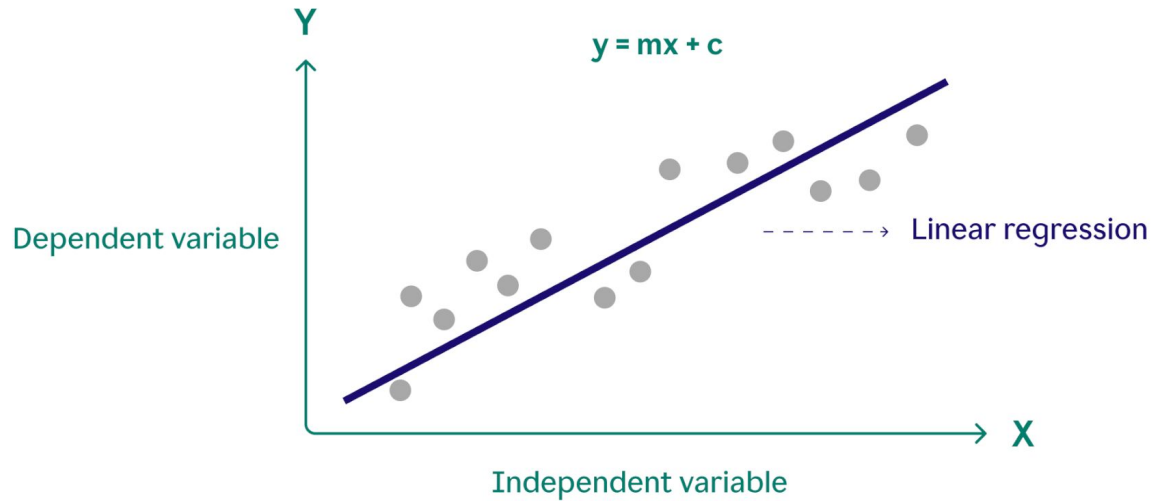


Training data

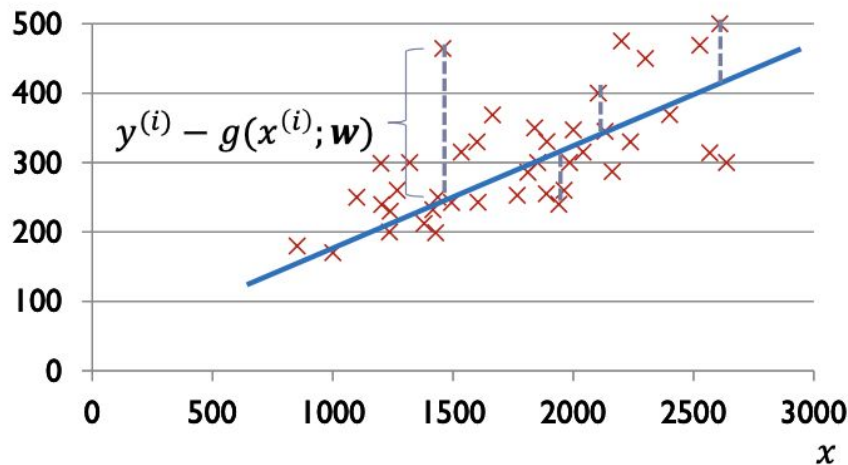
x_1	x_2	y	
0.9	2.3	1	—
3.5	2.6	1	—
2.6	3.3	1	—
2.7	4.1	1	—
1.8	3.9	1	—
6.5	6.8	-1	×
7.2	7.5	-1	×
7.9	8.3	-1	×
6.9	8.3	-1	×
8.8	7.9	-1	×
9.1	6.2	-1	×



Linear Regression

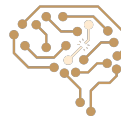


Sum of Squared Error (SSE)



Cost function:

$$\begin{aligned} J(\mathbf{w}) &= \sum_{i=1}^n (y^{(i)} - g(x^{(i)}; \mathbf{w}))^2 \\ &= \sum_{i=1}^n (y^{(i)} - w_0 - w_1 x^{(i)})^2 \end{aligned}$$

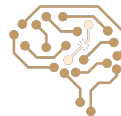


Gradient Descent

- Optimization algorithm to find $\mathbf{w}^* = \underset{\mathbf{w}}{\operatorname{argmin}} J(\mathbf{w})$
- Gradient descent: In each step, takes steps proportional to the negative of the gradient vector of the function at the current point:

$$\mathbf{w}^{t+1} = \mathbf{w}^t - \gamma_t \nabla J(\mathbf{w}^t)$$

- $J(\mathbf{w})$ decreases fastest if one goes from $\mathbf{w}^t$ in the direction of $-\nabla J(\mathbf{w}^t)$
- Assumption: $J(\mathbf{w})$ is defined and differentiable in a neighborhood of a point $\mathbf{w}^t$



Gradient Descent for SSE Cost Function

Minimize $J(\mathbf{w})$

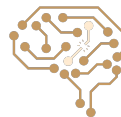
$$\mathbf{w}^{t+1} = \mathbf{w}^t - \eta \nabla_{\mathbf{w}} J(\mathbf{w}^t)$$

$J(\mathbf{w})$: Sum of squares error

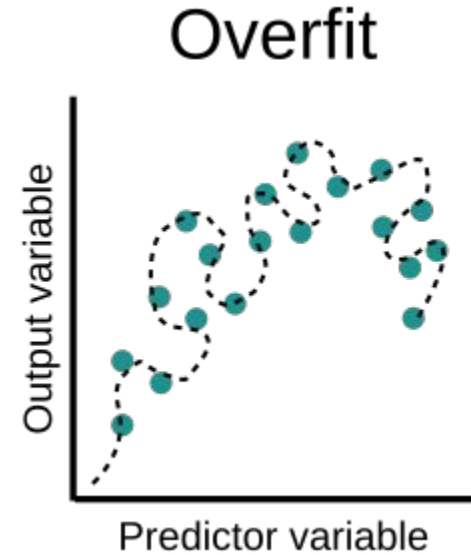
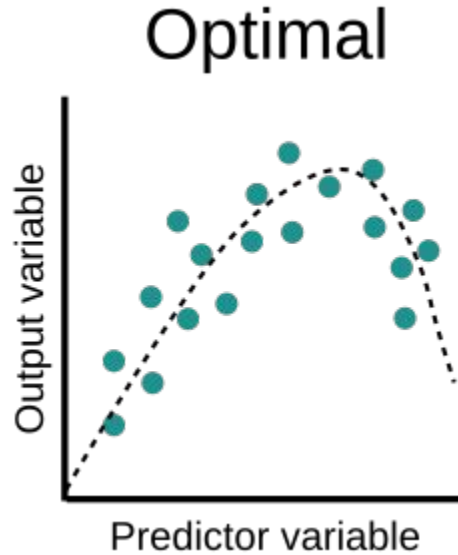
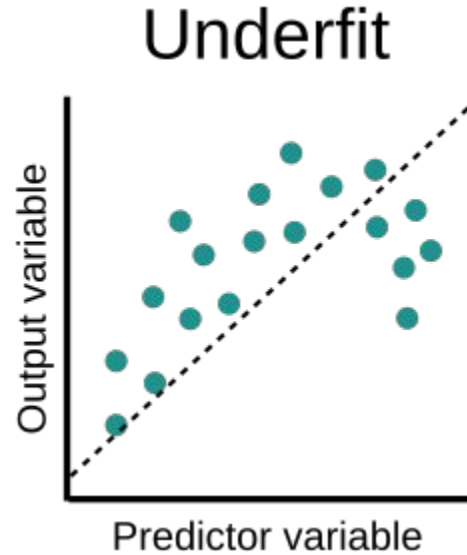
$$J(\mathbf{w}) = \sum_{i=1}^n \left(y^{(i)} - g(\mathbf{x}^{(i)}; \mathbf{w}) \right)^2$$

Weight update rule for $g(\mathbf{x}; \mathbf{w}) = \mathbf{w}^T \mathbf{x}$: $\mathbf{w} = \begin{bmatrix} w_0 \\ w_1 \\ \vdots \\ w_d \end{bmatrix}$ $\mathbf{x} = \begin{bmatrix} 1 \\ x_1 \\ \vdots \\ x_d \end{bmatrix}$

$$\mathbf{w}^{t+1} = \mathbf{w}^t + \eta \sum_{i=1}^n \left(y^{(i)} - \mathbf{w}^{tT} \mathbf{x}^{(i)} \right) \mathbf{x}^{(i)}$$



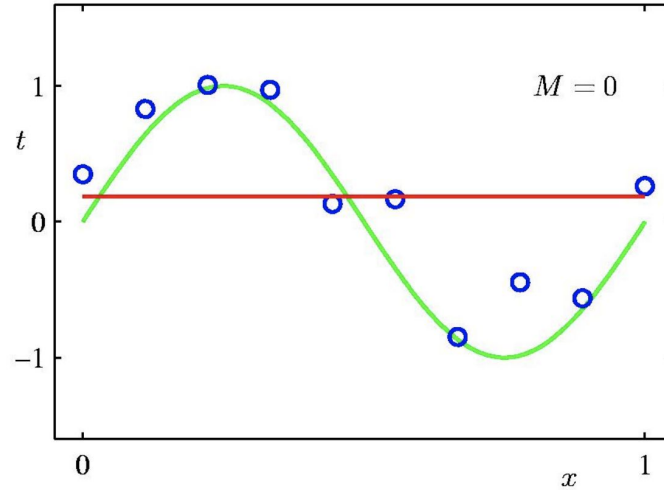
Three Possible Scenarios



Regression Example: Order

- Underfit!

0th Order Polynomial



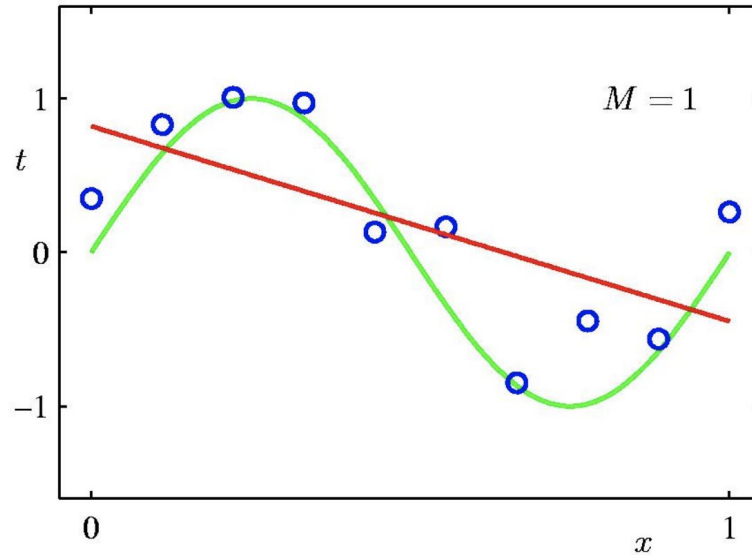
$n=10$



Regression Example: Order

- Underfit!

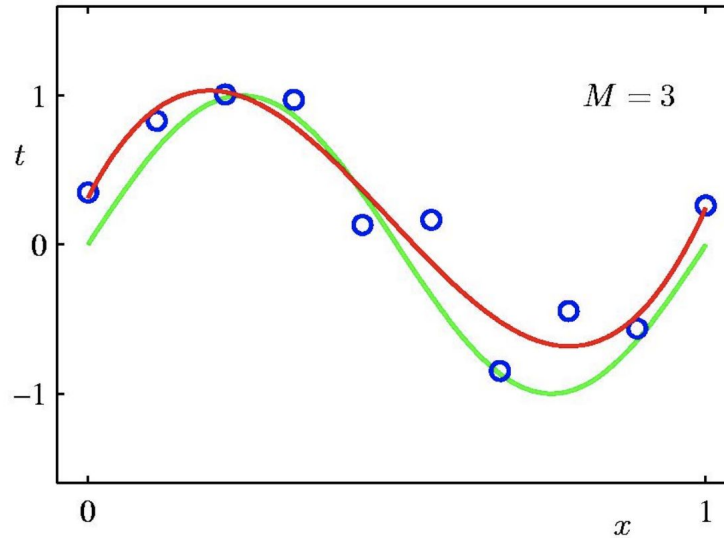
1st Order Polynomial



Regression Example: Order

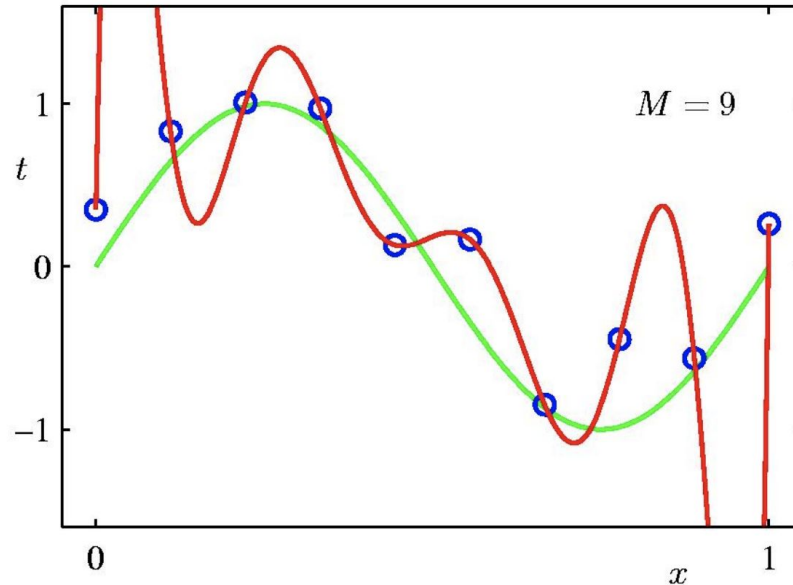
- Just Fit!

3rd Order Polynomial



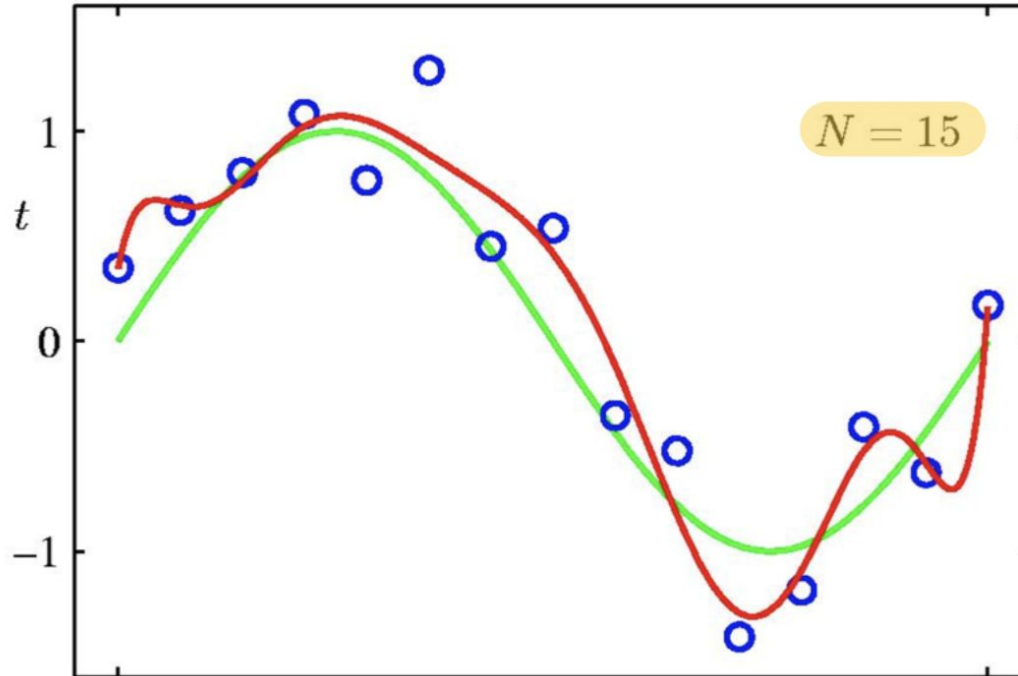
Regression Example: Order

- Overfit!
- ## 9th Order Polynomial



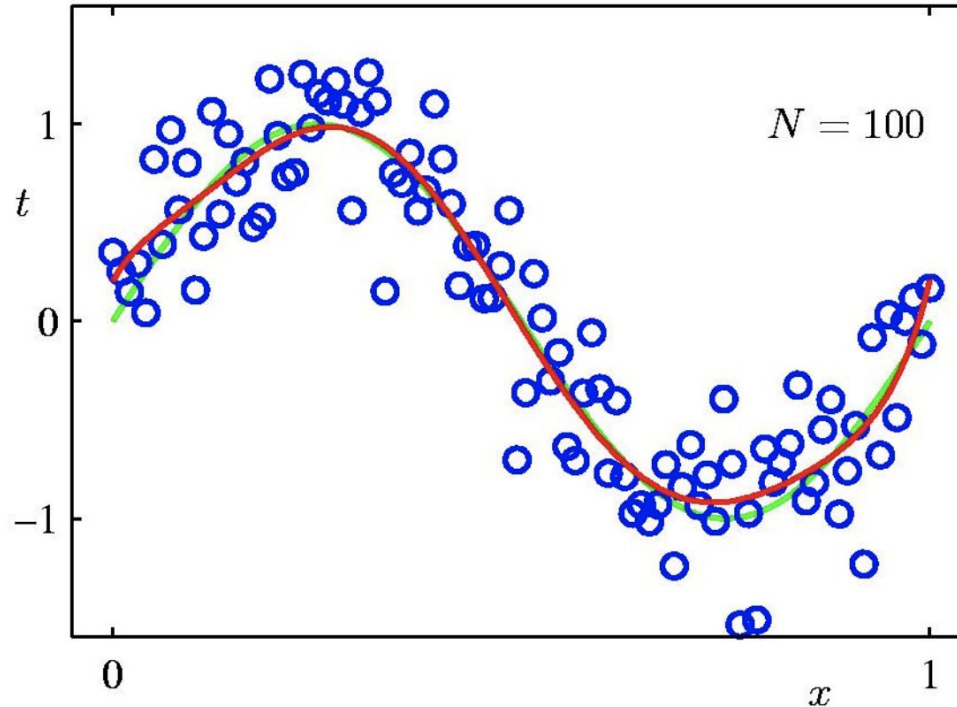
Regression Example: Dataset Size

- 9th order polynomial



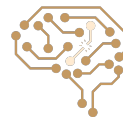
Regression Example: Dataset Size

9th order polynomial



Polynomial Coefficients

	$M = 0$	$M = 1$	$M = 3$	$M = 9$
w_0^*	0.19	0.82	0.31	0.35
w_1^*		-1.27	7.99	232.37
w_2^*			-25.43	-5321.83
w_3^*			17.37	48568.31
w_4^*				-231639.30
w_5^*				640042.26
w_6^*				-1061800.52
w_7^*				1042400.18
w_8^*				-557682.99
w_9^*				125201.43



Ridge Regression

Adds an **L2 regularizer** to Linear Regression

$$J_{\text{RR}}(\boldsymbol{\theta}) = J(\boldsymbol{\theta}) + \lambda \|\boldsymbol{\theta}\|_2^2$$

$$= \frac{1}{2} \sum_{i=1}^N (\boldsymbol{\theta}^T \mathbf{x}^{(i)} - y^{(i)})^2 + \lambda \sum_{k=1}^K \theta_k^2$$

prefers
parameters
close to zero

Bayesian interpretation: MAP estimation
with a **Gaussian prior** on the parameters



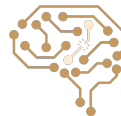
Lasso Regression

Adds an **L1 regularizer** to Linear Regression

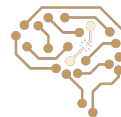
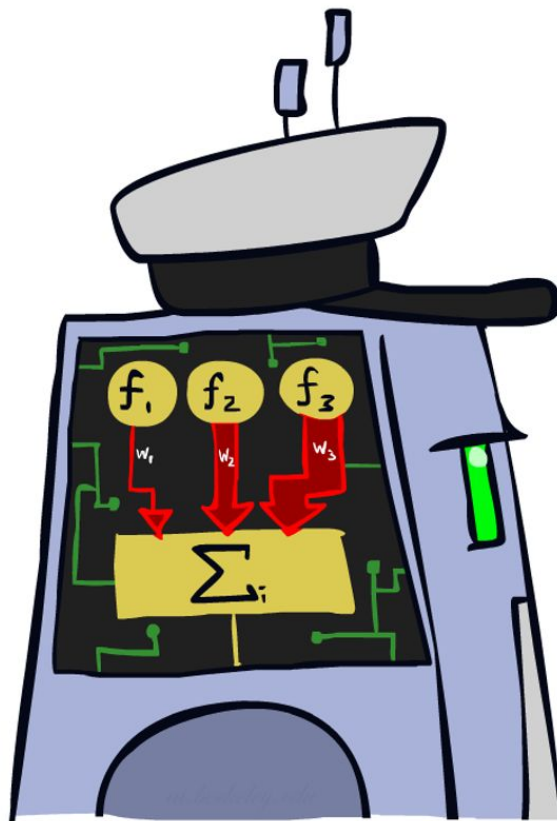
$$\begin{aligned} J_{\text{LASSO}}(\boldsymbol{\theta}) &= J(\boldsymbol{\theta}) + \lambda \|\boldsymbol{\theta}\|_1 \\ &= \frac{1}{2} \sum_{i=1}^N (\boldsymbol{\theta}^T \mathbf{x}^{(i)} - y^{(i)})^2 + \lambda \sum_{k=1}^K |\theta_k| \end{aligned}$$

yields sparse parameters (exact zeros)

Bayesian interpretation: MAP estimation with a **Laplace prior** on the parameters

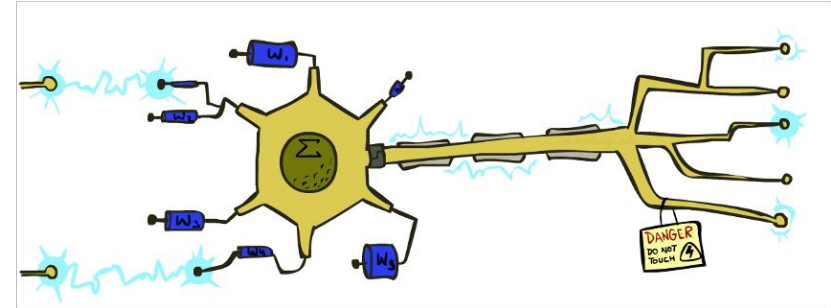
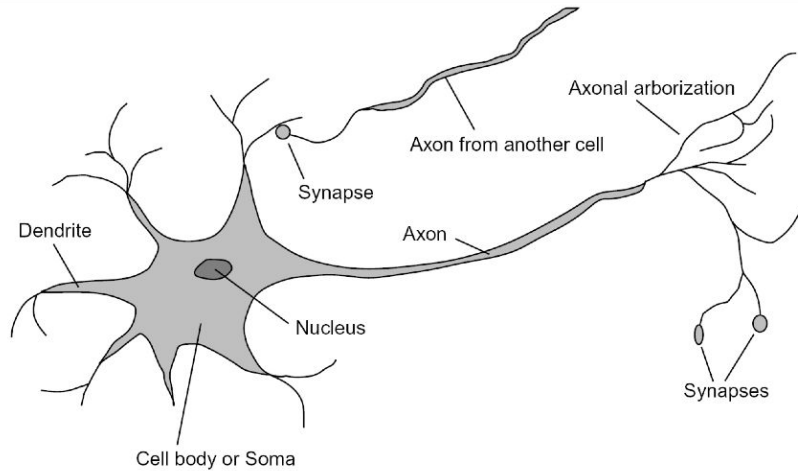


Linear Classifiers

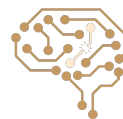
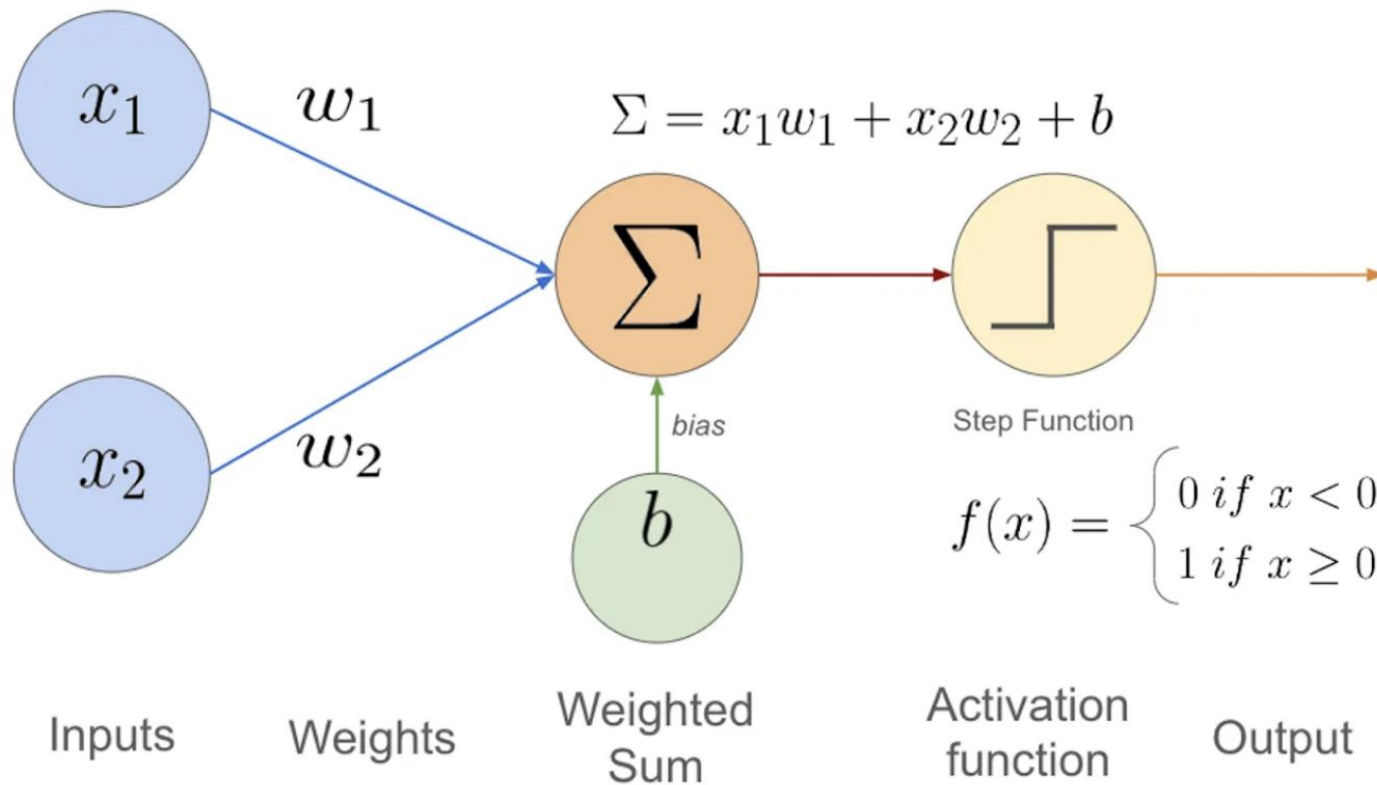


Some (Simplified) Biology

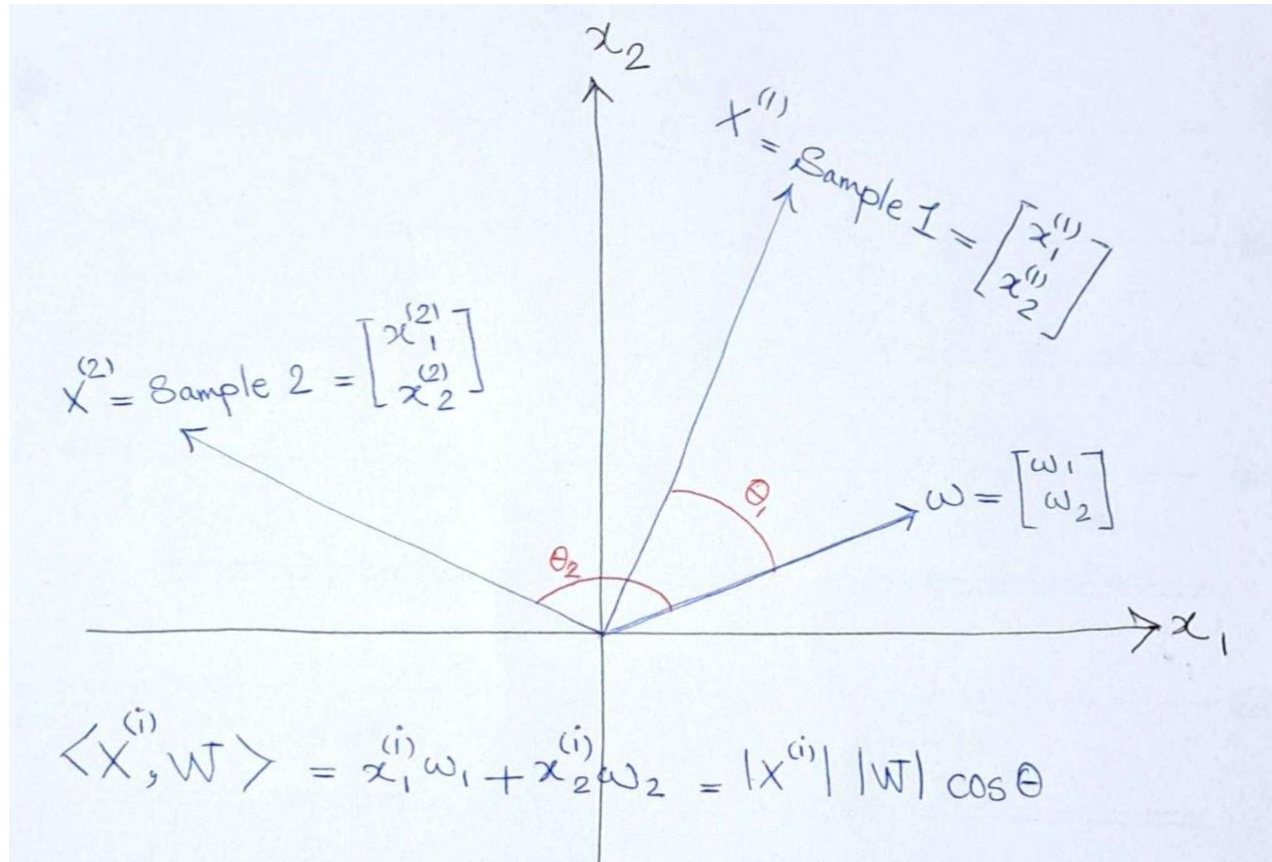
- Very loose inspiration: human neurons



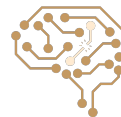
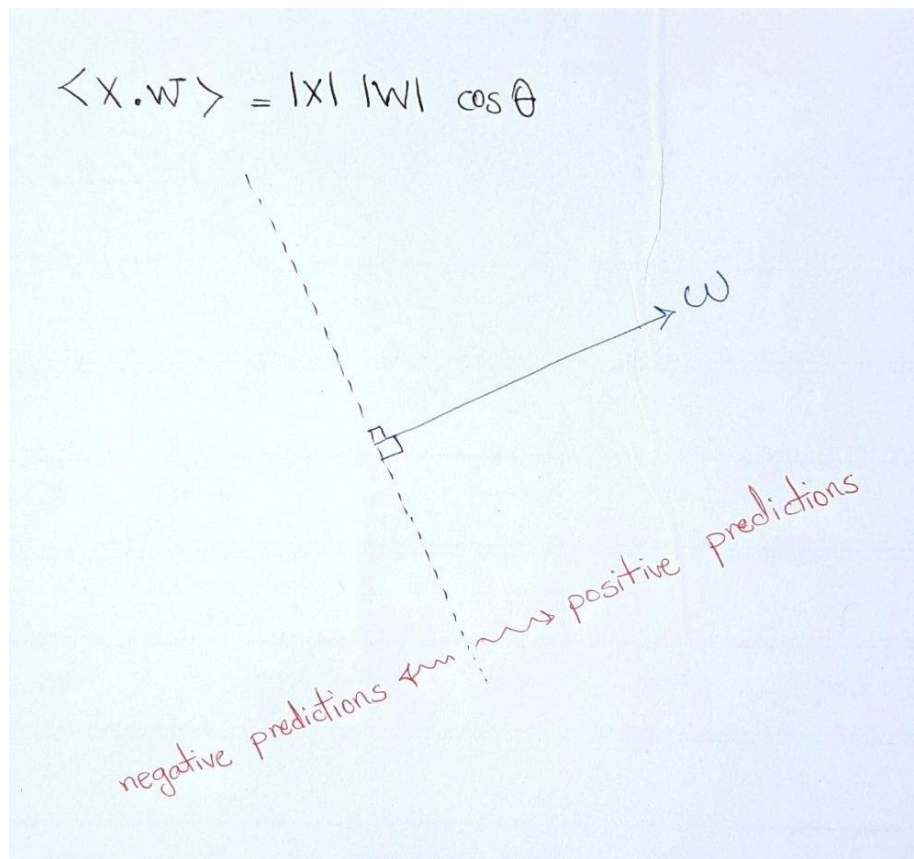
Perceptron



Geometric Interpretation



Geometric Interpretation (cont.)



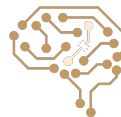
What About Cost Function?

- SSE seems suitable for Perceptron

$$SSE = \sum (y - y')^2$$

- But we can **not** use Gradient Descent algorithm to find weights. Why?

$$\mathbf{w}^{t+1} = \mathbf{w}^t - \eta \nabla_{\mathbf{w}} J(\mathbf{w}^t)$$



Learning Algorithm of Perceptron

- ▶ Given N training instances $(\mathbf{x}^{(1)}, y^{(1)}), (\mathbf{x}^{(2)}, y^{(2)}), \dots, (\mathbf{x}^{(N)}, y^{(N)})$
 - ▶ $y^{(n)} = +1$ or -1

- Initialize $\mathbf{w}$
- Cycle through the training instance
- While more classification errors

- For $i = 1 \dots N_{train}$
 - $\hat{y}^{(i)} = \text{sign}(\mathbf{w}^T \mathbf{x}^{(i)})$
 - If $\hat{y}^{(i)} \neq y^{(i)}$
 - $\mathbf{w} = \mathbf{w} + y^{(i)} \mathbf{x}^{(i)}$

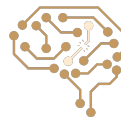
If instance misclassified:

– If instance is positive class

$$\mathbf{w} = \mathbf{w} + \mathbf{x}$$

– If instance is negative class

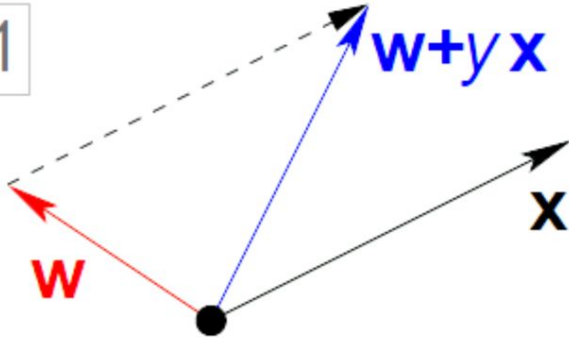
$$\mathbf{w} = \mathbf{w} - \mathbf{x}$$



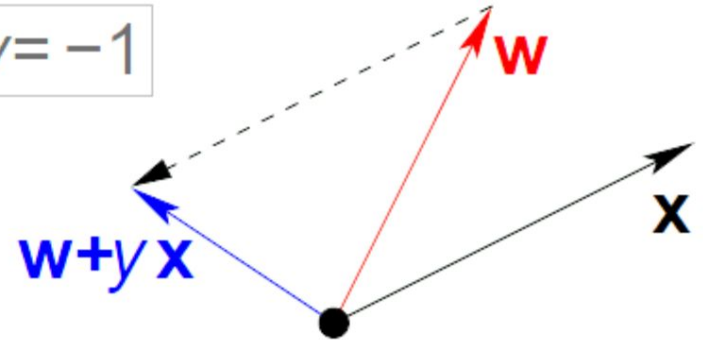
Geometric Interpretation

- Perceptron Learning when misclassified data has label $y=+1$ or $y=-1$

$$y = +1$$



$$y = -1$$



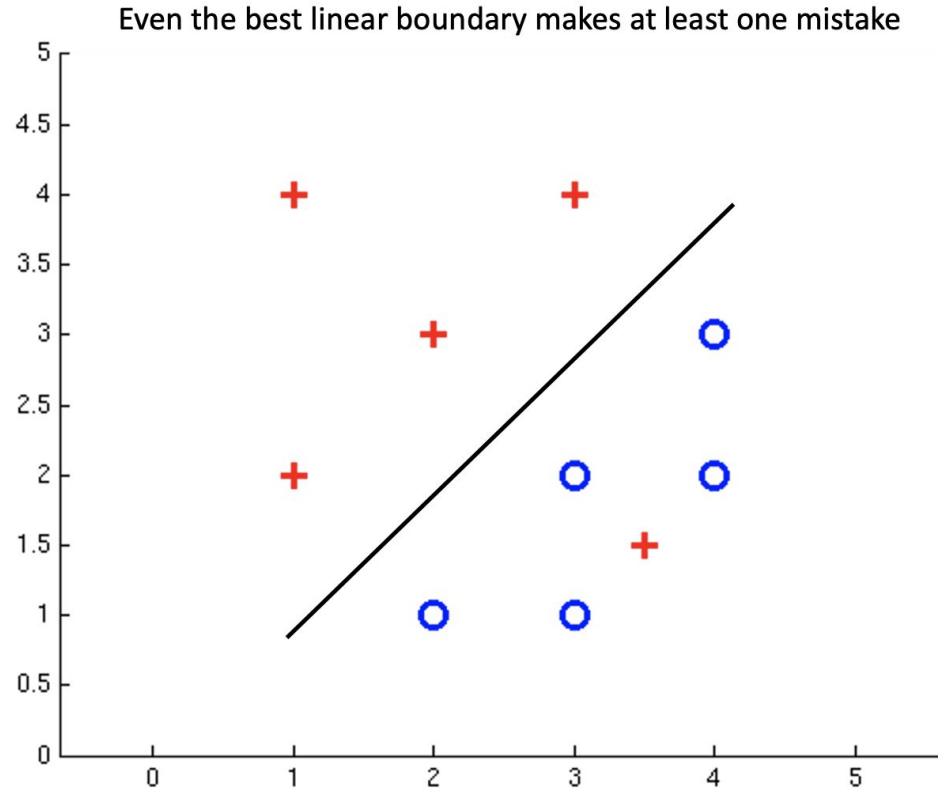
Perceptron Convergence

If the training data $D = \{(x_1, y_1), \dots, (x_N, y_N)\}$ is **linearly separable** with margin γ by a unit norm hyperplane w_* ($\|w_*\| = 1$) with $b = 0$,

Then **perceptron training converges after** $\frac{R^2}{\gamma^2}$ **errors** during training (assuming $\|x\| < R$ for all x).

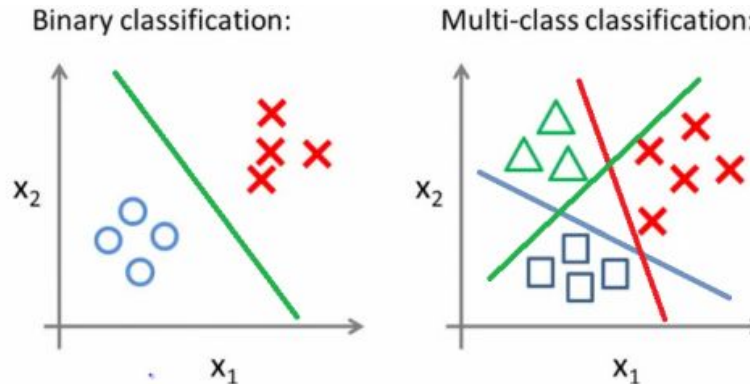


Non-Linearly Separable Case



Multi-class Perceptron

- In multiple classes classification, we have a weight vector for each class
- Score of a class y : $\langle W^{(y)}, X \rangle$
- Prediction class: highest score $y = \arg \max_y \langle W^{(y)}, X \rangle$



Multi-class Perceptron: Learning

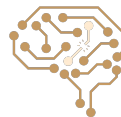
- Start with all weights = 0
- Pick up training examples one by one
- Predict the class of sample:

$$y = \arg \max_y \langle W^{(y)}, X \rangle$$

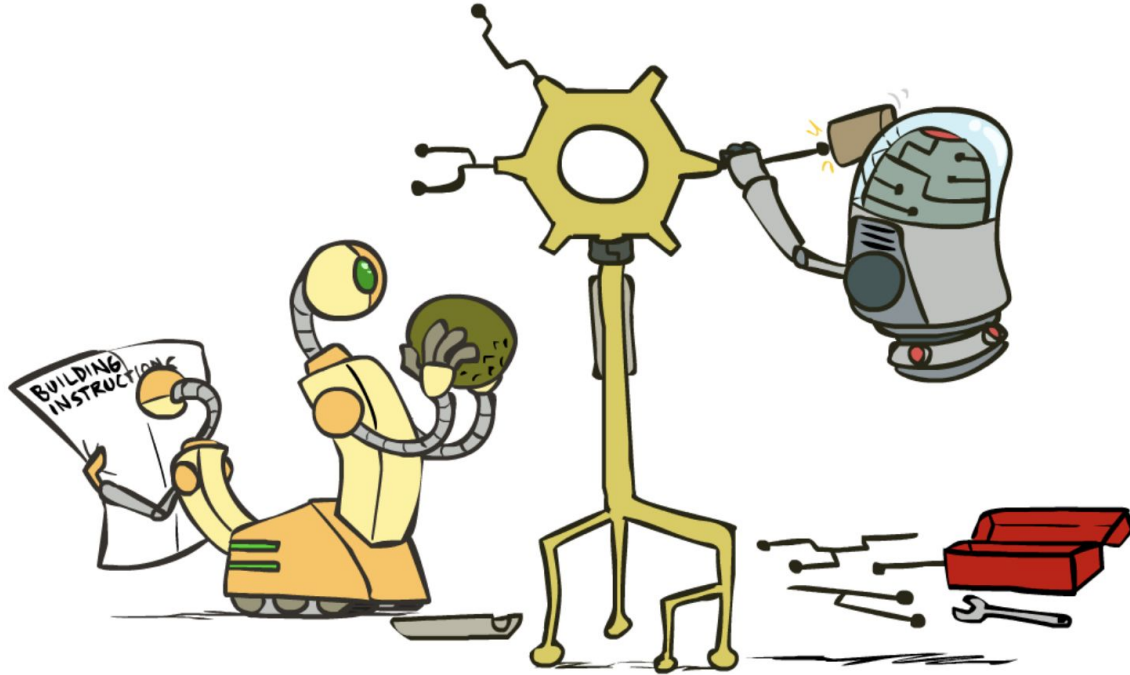
- If correct, no change!
- If wrong: lower score of wrong answer, raise score of right answer

$$W^{(y)} = W^{(y)} - X$$

$$W^{(y^*)} = W^{(y^*)} + X$$

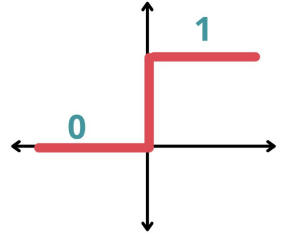


Improving the Perceptron: Logistic Regression



Downsides of Perceptron

- The activation function in perceptron is step function
- So, the predictions are binary (e.g, 0 or 1)
- What about a probabilistic prediction (e.g, $[0, 1]$)
 - 0 means the sample completely belongs to class 0
 - 0 means the sample completely belongs to class 1
 - 0.7 means the sample belongs to class 1 by 70%
- Also, the activation function of perceptron was not differentiable, so we could not use gradient descent

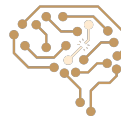


Sigmoid: Activation Function for Probabilistic Decisions

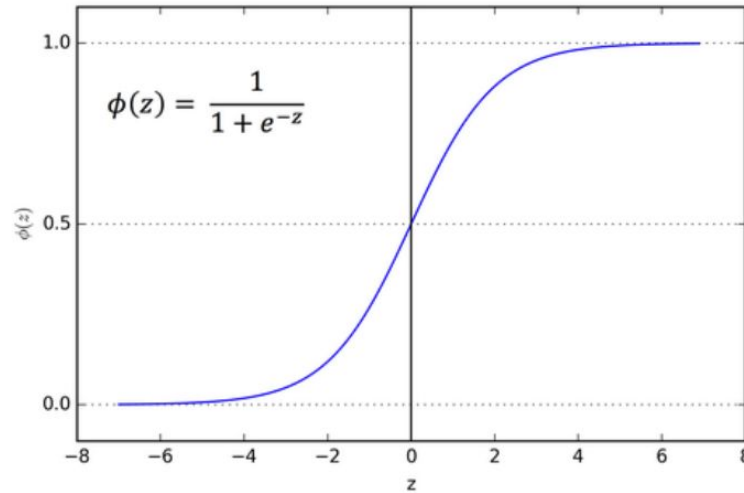
- Scoring: $z = \langle w, x \rangle$
- If $z = w \cdot f(x)$ very positive $\Rightarrow$ want probability going to 1
- If $z = w \cdot f(x)$ very negative $\Rightarrow$ want probability going to 0
- Sigmoid function (logistic function):

$$\phi(z) = \frac{1}{1 + e^{-z}}$$

- We can use sigmoid as an activation Function (instead of step function)



Sigmoid Activation Function



- Is SSE a good choice for cost function?

$$SSE = \sum (y - y')^2$$



Cost Function

$$J(\mathbf{w}) = \sum_{i=1}^n -y^{(i)} \log(\sigma(\mathbf{w}^T \mathbf{x}^{(i)})) - (1 - y^{(i)}) \log(1 - \sigma(\mathbf{w}^T \mathbf{x}^{(i)}))$$

$J(\mathbf{w})$ is convex w.r.t. parameters.

Since $y = 1$ or $y = 0$

$$\text{Loss}(y, \sigma(\mathbf{x}; \mathbf{w})) = \begin{cases} -\log(\sigma(\mathbf{x}; \mathbf{w})) & \text{if } y = 1 \\ -\log(1 - \sigma(\mathbf{x}; \mathbf{w})) & \text{if } y = 0 \end{cases}$$

$\Rightarrow$



Logistic Regression

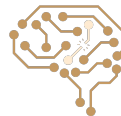
- The activation function is Sigmoid, which is differentiable
- The cost function: $\sum_{i=1}^n -y^{(i)}\log(\sigma(\mathbf{w}^T \mathbf{x}^{(i)})) - (1 - y^{(i)})\log(1 - \sigma(\mathbf{w}^T \mathbf{x}^{(i)}))$
- Gradient Descent:

$$\mathbf{w}^{t+1} = \mathbf{w}^t - \eta \nabla_{\mathbf{w}} J(\mathbf{w}^t)$$

$$\nabla_{\mathbf{w}} J(\mathbf{w}) = \sum_{i=1}^n (\sigma(\mathbf{w}^T \mathbf{x}^{(i)}) - y^{(i)}) \mathbf{x}^{(i)}$$

- It is similar to gradient update for the SSE loss in Linear Regression!

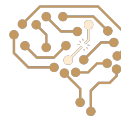
$$\nabla_{\mathbf{w}} J(\mathbf{w}) = \sum_{i=1}^n (\mathbf{w}^T \mathbf{x}^{(i)} - y^{(i)}) \mathbf{x}^{(i)}$$



Multiclass Logistic Regression

- We have a weight vector for each class
- Score of a class y : $z^{(y)} = \langle W^{(y)}, X \rangle$
- We can not use sigmoid anymore
- Softmax: an extension of sigmoid for multi-class scenarios

$$\underbrace{z_1, z_2, z_3}_{\text{original activations}} \rightarrow \underbrace{\frac{e^{z_1}}{e^{z_1} + e^{z_2} + e^{z_3}}, \frac{e^{z_2}}{e^{z_1} + e^{z_2} + e^{z_3}}, \frac{e^{z_3}}{e^{z_1} + e^{z_2} + e^{z_3}}}_{\text{softmax activations}}$$

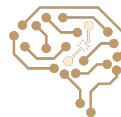


Multiclass Logistic Regression (cont.)

- Loss Function (for one sample):

$$L_{CE}(\hat{y}, y) = - \sum_{k=1}^K y_k \log(\hat{y}_k)$$

- Note that for a specific value of k , y_k is 1, and for all others it is 0



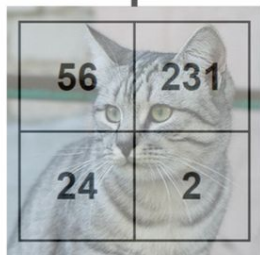
Softmax Example

$$\mathbf{x} = \begin{bmatrix} x_1 \\ \vdots \\ x_4 \end{bmatrix}$$

$$\mathbf{W}^T = \begin{bmatrix} \mathbf{w}_1 \\ \mathbf{w}_2 \\ \mathbf{w}_3 \end{bmatrix}_{3 \times 4}$$

$$\mathbf{w}_0 = \begin{bmatrix} b_1 \\ b_2 \\ b_3 \end{bmatrix}$$

Stretch pixels into column



Input image

0.2	-0.5	0.1	2.0
1.5	1.3	2.1	0.0
0	0.25	0.2	-0.3

$\mathbf{W}^T$

56
231
24
2

+

1.1
3.2
-1.2

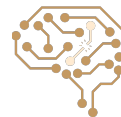
=

-96.8
437.9
61.95

Cat score

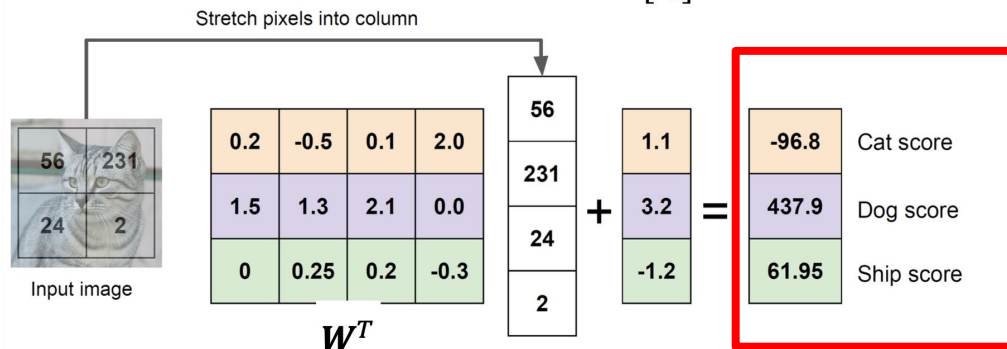
Dog score

Ship score



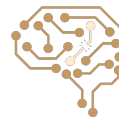
Softmax Example (cont.)

$$x = \begin{bmatrix} x_1 \\ \vdots \\ x_4 \end{bmatrix} \quad W^T = \begin{bmatrix} w_1 \\ w_2 \\ w_3 \end{bmatrix}_{3 \times 4} \quad w_0 = \begin{bmatrix} b_1 \\ b_2 \\ b_3 \end{bmatrix}$$



These are z_1, z_2, z_3 scores (logits), to convert them to probabilities, use the below expression

$$\underbrace{z_1, z_2, z_3} \rightarrow \underbrace{\frac{e^{z_1}}{e^{z_1} + e^{z_2} + e^{z_3}}, \frac{e^{z_2}}{e^{z_1} + e^{z_2} + e^{z_3}}, \frac{e^{z_3}}{e^{z_1} + e^{z_2} + e^{z_3}}}$$



Next Session: Neural Networks

